

Observability Preview: Prometheus, Grafana, cAdvisor, Loki

이 preview는 Day 4 본수업의 필수 범위가 아니다. 목적은 docker logs와 docker stats를 배운 뒤, 같은 관찰을 Compose 기반 stack으로 확장하면 어떤 모양이 되는지 미리 보는 것이다.

핵심 메시지:

logs: 무슨 일이 있었는가
metrics: 시간에 따라 상태가 어떻게 변했는가
inspect/exec: 지금 적용된 설정과 내부 상태가 무엇인가

Architecture

Service	Port	역할
sample-web	18085	nginx test traffic 대상
log-generator	none	일반 log를 계속 출력하는 sample container
load-generator	none	sample-web에 반복 HTTP 요청
cpu-spike	none	선택 profile, CPU spike 체험
cadvisor	18086	Docker container CPU/memory/network metrics 노출
prometheus	19090	cAdvisor metrics scrape
loki	13100	container log 저장
promtail	none	Docker container log를 Loki로 전달
grafana	13000	Prometheus metrics와 Loki logs 시각화

Run

먼저 OS별로 Docker log 경로를 확인한다. cAdvisor와 Promtail은 container 내부 상태를 보기 위해 Docker engine의 실제 data root를 mount해야 한다.

환경	권장 확인
WSL/Linux	<code>docker info --format '{{.DockerRootDir}}'</code> 결과를 DOCKER_ROOT_DIR로 export
macOS Docker Desktop	Docker engine이 Linux VM 안에서 동작하므로 경로 mount가 제한될 수 있음. Grafana/Prometheus UI 확인을 우선하고, log 수집이 안 되면 <code>docker compose logs</code> 로 대체

```
cd /mnt/d/paperclip/week2/day4/labs/observability-preview
export DOCKER_ROOT_DIR="$(docker info --format '{{.DockerRootDir}}')"
docker compose config
docker compose --profile load config
docker compose up -d
docker compose ps
```

Expected:

```
grafana      running
prometheus   running
cadvisor     running
loki         running
promtail     running
sample-web   running
load-generator running
```

Generate traffic and logs

```
for i in 1 2 3 4 5; do curl -I http://localhost:18085 || true; sleep 1; done
docker compose logs sample-web --tail 20
docker compose logs log-generator --tail 20
```

Expected:

```
HTTP/1.1 200 OK
level=info service=log-generator event=heartbeat
```

Impact drill: stats snapshot vs metrics timeline

`docker stats`는 지금 이 순간의 값이다. Prometheus/Grafana는 시간이 지나며 값이 어떻게 변했는지 본다. 차이를 느끼기 위해 CPU spike container를 profile로 켜다.

```
docker compose --profile load up -d cpu-spike
docker stats --no-stream
sleep 20
curl -s 'http://localhost:19090/api/v1/query?
query=rate(container_cpu_usage_seconds_total%5B1m%5D)' | grep cpu-spike ||
true
```

Expected:

```
cpu-spike
CPU %
```

Grafana Explore > Prometheus에서 다음 query를 넣는다.

```
rate(container_cpu_usage_seconds_total[1m])
```

판단:

관찰 도구	보이는 것	한계
<code>docker stats --no-stream</code>	지금 순간 CPU/MEM	이전 1분 동안의 추세를 보기 어렵다
Prometheus query	시간 window의 변화	어떤 log line 때문에 튀었는지는 별도 log 확인 필요
Grafana Explore	metrics와 logs를 한 화면에서 탐색	dashboard가 원인을 자동으로 고쳐주지는 않는다

CPU spike를 멈춘다.

```
docker compose stop cpu-spike
```

Check Prometheus

```
curl -s http://localhost:19090/-/ready
curl -s 'http://localhost:19090/api/v1/query?query=up' | grep cadvisor
```

Expected:

```
Prometheus Server is Ready.
cadvisor
```

Prometheus UI:

```
http://localhost:19090
```

Example queries:

```
up
container_memory_usage_bytes
```

```
rate(container_cpu_usage_seconds_total[1m])
rate(container_network_receive_bytes_total[1m])
```

Check Grafana

Open:

```
http://localhost:13000
```

Login:

```
admin / practice-only
```

확인할 것:

화면	확인
Connections > Data sources	Prometheus, Loki datasource가 있는가
Explore > Prometheus	up, container_memory_usage_bytes query가 되는가
Explore > Loki	{job="docker"} log query가 되는가

Check Loki logs

Promtail이 Docker log file을 읽을 수 있는 환경이면 Grafana Explore에서 다음 query가 동작한다.

```
{job="docker"}
```

curl로 직접 확인할 때는 log query가 시간 범위를 필요로 하므로 query_range를 사용한다. query endpoint에 같은 LogQL을 넣으면 instant query라서 실패할 수 있다.

WSL/Linux:

```
now_ns=$(date +%s%N)
start_ns=$((now_ns - 3000000000000))

curl -G -s 'http://localhost:13100/loki/api/v1/query_range' \
  --data-urlencode 'query={job="docker"}' \
  --data-urlencode "start=$start_ns" \
  --data-urlencode "end=$now_ns" \
  --data-urlencode 'limit=5'
```

macOS:

```
end_time=$(date -u +"%Y-%m-%dT%H:%M:%SZ")
start_time=$(date -u -v-5M +"%Y-%m-%dT%H:%M:%SZ")

curl -G -s 'http://localhost:13100/loki/api/v1/query_range' \
  --data-urlencode 'query={job="docker"}' \
  --data-urlencode "start=$start_time" \
  --data-urlencode "end=$end_time" \
  --data-urlencode 'limit=5'
```

Expected:

```
"status": "success"
"job": "docker"
```

환경에 따라 Docker Desktop/WSL의 log path mount가 제한될 수 있다. 이 경우에도 Day 4의 기본 log 확인은 다음 명령으로 계속 가능하다.

```
docker compose logs sample-web
docker compose logs log-generator
```

Troubleshooting hints

Symptom	First check	Likely cause
docker-credential-desktop.exe not found	docker compose up -d output	WSL이 Windows Docker credential helper를 찾지 못함
Grafana login fails	docker compose logs grafana	password typo or old grafana-data volume
Prometheus has no cAdvisor target	docker compose logs prometheus	scrape target unavailable
cAdvisor container exits	docker compose logs cadvisor	host mount/device restriction
Loki has no logs	docker compose logs promtail	Docker log path mount restriction
port already allocated	docker compose ps, docker ps	local port conflict
CPU spike가 계속 남음	docker compose ps	cpu-spike profile container stop 필요

WSL/Linux troubleshooting

증상	확인 명령	조치
docker-credential-desktop.exe 오류	cat ~/.docker/config.json	실습 중에는 임시 config로 우회: mkdir -p /tmp/paperclip-docker-config && printf '{}\n' > /tmp/paperclip-docker-config/config.json, 이후 DOCKER_CONFIG=/tmp/paperclip-docker-config docker compose up -d
cAdvisor가 /var/lib/docker를 못 읽음	docker info --format '{{.DockerRootDir}}'	export DOCKER_ROOT_DIR="\$(docker info --format '{{.DockerRootDir}}')\" 후 다시 docker compose up -d
port is already allocated	docker ps --format 'table {{.Names}}\t{{.Ports}}'	충돌 중인 container를 멈추거나 compose port를 변경. 이 lab은 cAdvisor를 18086으로 사용
Loki API에서 log queries are not supported as an instant query type	호출한 URL 확인	/loki/api/v1/query가 아니라 /loki/api/v1/query_range 사용

macOS Docker Desktop troubleshooting

증상	확인 명령	조치
date +%s%N 결과가 이상함	date +%s%N	macOS용 date -u -v-5M 예시를 사용
Promtail이 Docker log를 못 읽음	docker compose logs promtail	Docker Desktop VM 내부 log path 제한일 수 있음. Grafana/Loki가 비어 있으면 docker compose logs sample-web, docker compose logs log-generator로 일반 로그 확인
cAdvisor가 device/mount 오류로 종료	docker compose logs cadvisor	Docker Desktop 환경 제약일 수 있음. 이 경우 docker stats와 Prometheus/Grafana UI 확인을 우선하고, metrics target 실패 원인을 토론 포인트로 삼음
Grafana는 뜨는데 Prometheus target이 down	curl -s 'http://localhost:19090/api/v1/query?query=up'	cAdvisor scrape 실패 가능성. Docker Desktop VM mount/device 제약 여부 확인

Cleanup

```
docker compose down
```

Data reset까지 필요할 때만 실행한다.

```
docker compose down -v
```

down -v는 Grafana/Loki volume을 삭제한다. 다음 실행 때 datasource는 provisioning으로 다시 생성되지만, 사용자가 만든 dashboard는 사라질 수 있다.